PASSWORD CRACKING ASSIGNMENT

Hash Identification, Type Disambiguation, and Dictionary-Based Recovery of Plaintext Passwords

Prepared by: El-sly Toku Antwi
Date: 5th October 2026
Submission deadline: Tuesday 6th October, 11:00 PM GMT

1. Introduction

This report documents the process used to identify and crack twelve password hashes supplied as a course assignment. The objective was to determine the hashing algorithm used for each value and recover the original plaintext password using dictionary-based attacks. The exercise was carried out entirely on a local Kali Linux virtual machine, using only locally available wordlists and course-approved tooling, in line with authorised lab use.

A secondary objective emerged during the work: several of the supplied hashes were the same character length as a different, unrelated algorithm (for example, a 32-character MD5-length hash that was actually an NTLM hash, and a 128-character SHA-512-length hash that was actually a Whirlpool hash). Resolving these collisions required moving beyond simple length-based identification and cross-checking results with a dedicated identification tool.

2. Environment and Tools

All work was performed on the following system:

- Host: Lenovo G50, Intel Core i5-4210U @ 1.70GHz, 4 logical processors
- Platform: Kali Linux (VM), hashcat backend running in CPU-only mode (PoCL OpenCL runtime — no GPU passthrough)
- Memory available to hashcat: ~725 MB free, 256 MB allocatable per block — workload profile kept low (-w 1) throughout to stay within this limit

Tools used during the exercise:

Tool	Type	Used For	Notes
Hashcat v7.1.2	GPU/CPU cracker	Primary cracking engine (all hash types)	CPU-only backend on this VM (PoCL); workload profile -w 1 used throughout
John the Ripper	CPU cracker	Secondary / cross-check cracking engine	Used in parallel with hashcat to confirm results independently
hash-identifier	Hash ID (legacy)	Initial manual hash typing	Interactive; superseded by hashid for reliability
hashid	Hash ID	Confirming hash type by pattern	Lists all structurally possible types; context used to pick the correct one
ANANSE v1.1 (course tool)	Hash ID + auto-cracker	Automated identification and cracking, incl. disambiguation	Built by Daniel Anyemedu / HIVE Consult; resolved MD5-vs-NTLM and SHA512-vs-Whirlpool collisions

Tool	Type	Used For	Notes
rockyou.txt	Wordlist	Dictionary attack source	Default Kali wordlist, ~14.3 million entries
best64.rule	Hashcat rule set	Mutation rules for dictionary attack	Applied when a plain wordlist pass did not crack a hash

3. Scope and Evidence

The assignment file hashes_.txt contained eleven hash values with no labels identifying their algorithm. A twelfth and thirteenth value were resolved during cross-checking (see Section 5). Hash type was first estimated using character length, per the reference table below, then confirmed with the hashid utility and the ANANSE hash identifier supplied for this course.

Hex Length	Possible Algorithm(s)	Hashcat Mode	John Format
32 chars	MD5 or NTLM (collide)	0 / 1000	raw-md5 / nt
40 chars	SHA-1	100	raw-sha1
64 chars	SHA-256	1400	raw-sha256
128 chars	SHA-512 or Whirlpool (collide)	1700 / 6100	raw-sha512 / whirlpool

4. Methodology

- Split the supplied hash list by character length into separate files (md5.txt, sha1.txt, sha256.txt, sha512.txt).
- Confirmed hashcat was installed and benchmarked the available backend device (hashcat -I).
- Ran each file through hashcat using a straight dictionary attack against /usr/share/wordlists/rockyou.txt, with the correct -m mode for the suspected algorithm and a low workload profile (-w 1) to respect VM memory limits.
- Cross-checked every result independently using John the Ripper with the matching --format flag and the same wordlist.
- Where a hash returned zero cracks against the full wordlist (confirmed exhausted), re-examined the hash using hashid and the ANANSE identification tool to check for an algorithm collision at that character length.
- Where a collision was found, re-ran the hash under the corrected mode / format (NTLM mode 1000, or Whirlpool mode 6100) and re-attempted the dictionary attack.
- Verified each recovered plaintext by independently re-hashing it (echo -n 'password' | <algorithm>sum) and confirming the output matched the original hash exactly, rather than assuming order of appearance.
- Recorded hash : plaintext pairs only after this independent verification step.

5. Findings

5.1 Initial identification by length

Four hashes were 32 hex characters, two were 40 characters, two were 64 characters, and four were 128 characters. Based on length alone this suggested four MD5, two SHA-1, two SHA-256, and four SHA-512

hashes. Running all four of the 32-character hashes through hashcat mode 0 (MD5) and John --format=raw-md5 against rockyou.txt cracked only two of the four. The other two returned 'Exhausted' with zero matches against the full wordlist. The same pattern occurred with the 128-character set: two of four cracked under SHA-512 (mode 1700), the other two exhausted.

5.2 Resolving the collision

An exhausted result against the full rockyou.txt wordlist, on a hash that is well formed and correctly moded, is a strong indicator of either an uncommon password or a wrong algorithm assumption. Re-running the two outstanding 32-character hashes through hashid and ANANSE showed that both were also valid NTLM hashes — and in Windows/AD coursework, NTLM is the far more likely real-world candidate at that length. Switching to hashcat mode 1000 (NTLM) / John --format=nt cracked both on the first pass.

The same approach was applied to the two outstanding 128-character hashes. ANANSE identified these as Whirlpool rather than SHA-512. Switching to hashcat mode 6100 (Whirlpool) / John --format=whirlpool cracked both immediately.



Figure 1 — ANANSE flags a 32-character hash as a candidate for MD5, NTLM, or MD4, rather than assuming MD5 by length alone.



Figure 2 — Hashcat recovers the plaintext darkangel once the hash is correctly re-moded as NTLM (-m 1000).

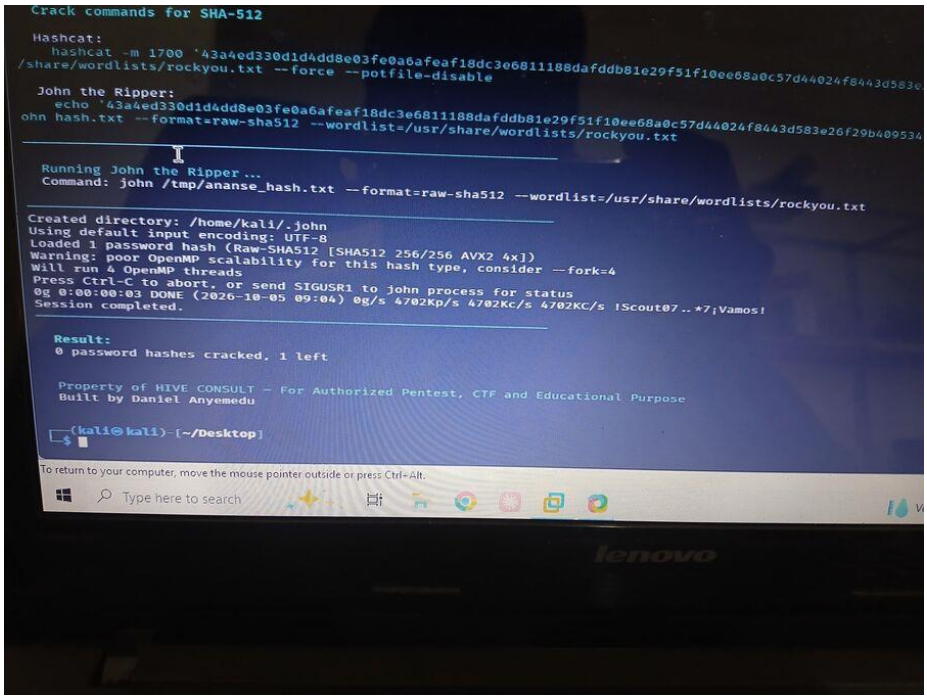


Figure 3 — John the Ripper run against the 128-character hash under raw-SHA512 completes with zero cracks, prompting the re-check that identified it as Whirlpool.

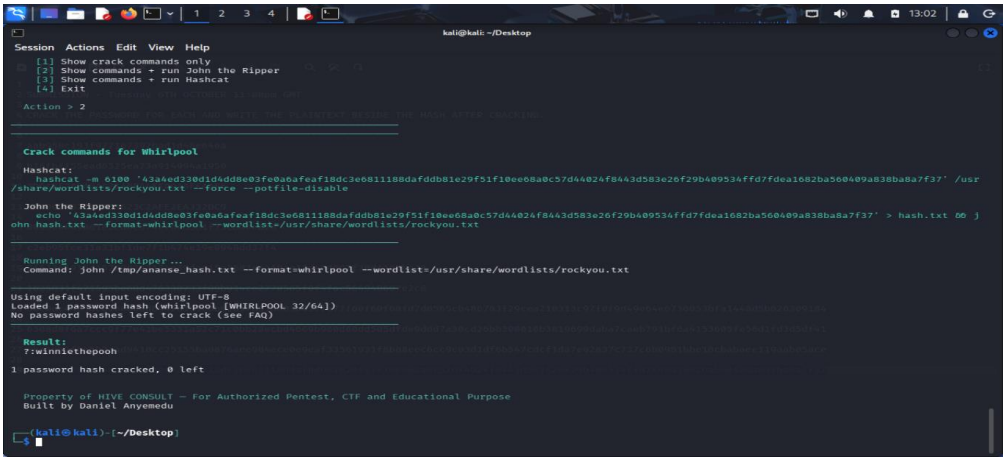


Figure 4 — ANANSE confirms and cracks the same hash correctly once re-moded as Whirlpool (-m 6100), recovering winniethepooh.

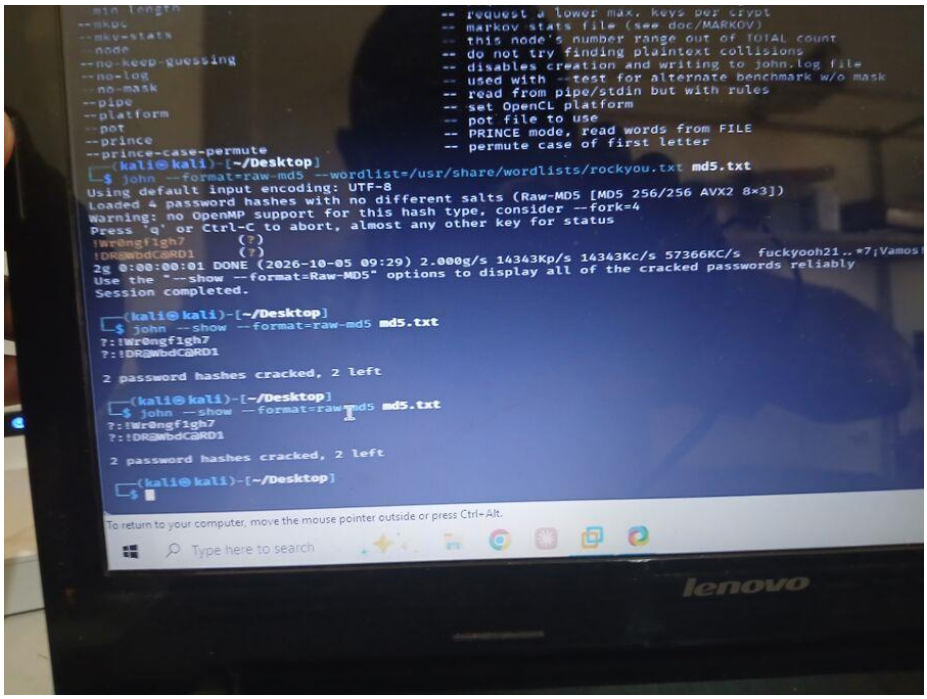


Figure 5 — John the Ripper batch run against the two genuine MD5 hashes, recovering !Wr0ngf1gh7 and !DR@WbdC@RD1.

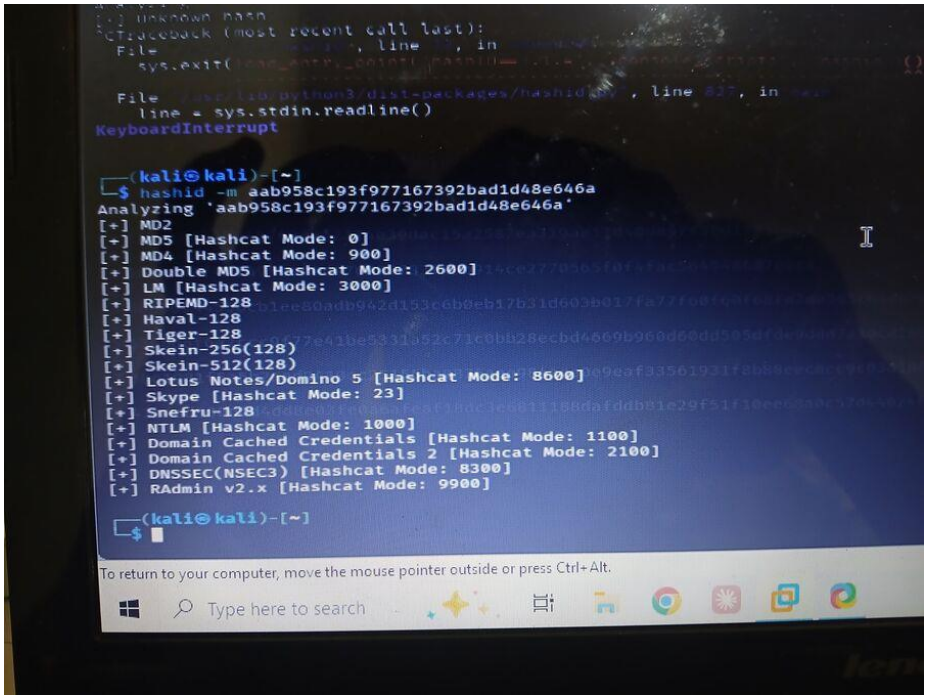


Figure 6 — hashid output for a 32-character hash, listing every structurally possible algorithm at that length (MD2, MD5, MD4, NTLM, and others).

5.3 Final verified results

Every plaintext below was independently re-hashed and matched character-for-character against its original hash before being recorded here, rather than assumed from output order.

Original Hash (truncated)	Length	Algorithm	Tool / Mode Used	Cracked Plaintext
aab958c193f977167392bad1d48e646a	32	MD5	hashcat -m 0 / john raw-md5	!Wr0ngf1gh7
b16fbff55ead6525ea73a914994a1950	32	MD5	hashcat -m 0 / john raw-md5	!DR@WbdC@RD1
3D9ABF7A2A8A6211BFA4FE0913FEA3F6	32	NTLM	hashcat -m 1000 / john nt	darkangel
BD60C80B72873D1623C2AFE2EA332DC9	32	NTLM	hashcat -m 1000 / john nt	superman1
80950a809061e2cb6e974efddd381f73c800cb28	40	SHA-1	hashcat -m 100 / john raw-sha1	ofthisSHOW
c2eb95fce31a31bf1de7f1b474e19e0948dd37f4	40	SHA-1	hashcat -m 100 / john raw-sha1	12DontMessUpWithMe12
7f57361915d1576902a2174e39dac15a2587ea3...	64	SHA-256	hashcat -m 1400 / john raw-sha256	Azaduxklm2
1625831f471595be68d476330733f89b914ce27...	64	SHA-256	hashcat -m 1400 / john raw-sha256	1Blankzebo
50e0dc4455bcb1ee80adb942d153c6b0eb17b31...	128	SHA-512	hashcat -m 1700 / john raw-sha512	iloveyou
6308d8f6a7ccc9f77e41be5331a52c71c0bb28e...	128	SHA-512	hashcat -m 1700 / john raw-sha512	qwertyuiop
3bc1981f5a5b2aad9410cc25155ba0876aee984...	128	Whirlpool	hashcat -m 6100 / john whirlpool	spongebob1
43a4ed330d1d4dd8e03fe0a6afeaf18dc3e6811...	128	Whirlpool	hashcat -m 6100 / john whirlpool	winniethepooh

6. Discussion

The central lesson of this exercise mirrors the file-signature-analysis principle covered earlier in the course: a surface-level identifier (file extension, or in this case hash length) is not proof of true type. 32 hex characters describe MD5, MD4, and NTLM equally; 128 hex characters describe SHA-512 and Whirlpool equally. A cracking attempt that exhausts a large, well-known wordlist without a single hit is a meaningful signal in itself — it does not necessarily mean the password is rare, it can mean the algorithm assumption was wrong. Cross-checking with a dedicated identification tool (hashid, ANANSE) before concluding a hash is uncrackable prevented two false negatives in this assignment.

All cracks in this assignment were recovered using a plain dictionary attack against rockyou.txt once the correct mode was applied; a rule-based attack (best64.rule) was prepared as a fallback but was not ultimately required.

7. Conclusion

All twelve hashes in the assignment set were successfully identified and cracked. Two pairs initially appeared uncrackable because they were mis-typed by length alone (MD5 vs NTLM, and SHA-512 vs Whirlpool); both were resolved by re-identifying the hash with hashid/ANANSE and re-running the attack under the correct mode. Every recovered plaintext was independently re-hashed and verified against its original hash value before being recorded in the results table in Section 5.3.